How to prompt Newly: ColdTrace team manual

Nov 11, 2026 · @Muedi

Use this manual to get the ColdTrace app out of Newly in small, checkable steps: set the rules once, prompt one screen at a time, and fix problems with targeted prompts instead of rebuilding.

1. What Newly is, and what to confirm first

Newly turns a chat description into a native iOS and Android app, so confirm four platform facts before the team commits to it.

- You describe the app in chat and an agent writes the code.
- You test it in a preview that runs on cloud simulators, so there is nothing to install.
- It can add a backend (Newly Backend) with sign-in, a PostgreSQL database, generated REST APIs, an API service for server code, file storage and logs.
- Every turn that changes files is saved as a version, and you can go back with Restore to here.
- Work is metered in credits by how much the agent does, not by how many messages you send.

The Newly documents do not state the framework or a web target, and two of them disagree on GitHub. Treat these as open until checked:

Confirm before relying on it	Why it matters	How to check
Framework: React Native or another	The brief asks for a React-based app. The docs only mention Expo and React Native in passing	Ask mode: "Which framework and language does this project use? Do not change anything."
Web output or tablet only	An operations console is used on desktops and tablets	Ask mode: "Can this project run on the web? Which screen sizes do you support?"
GitHub sync	One page says the code can sync both ways with a repository you own; the FAQ says projects do not sync to GitHub	Read the Connect GitHub page in the Newly docs, or ask Newly support, before depending on it

Confirm before relying on it	Why it matters	How to check
How the app reaches our Cloud Run API	The docs say the backend calls outside services, but do not show the request code	Ask mode: "Show how Newly Backend server code calls an external HTTPS API with a secret header." Then test it
Credits and plan	Credits are not capped, and repair loops cost credits	Check your plan under Account before the first long build

2. Set up before the first prompt

Put the rules and the brief into the project first, so every later prompt can be short.

1. Create a new project in Newly. Name it ColdTrace Operations.
2. Paste the standing rules as `AGENTS.md`. The ColdTrace build brief, section 2, contains the exact block. Ask the agent to create `AGENTS.md` at the project root with that text. Keep it under 20,000 characters; Newly reads it on every turn.
3. Attach the build brief to the first message so the agent can read the design system, screens and data contract.
4. Put the connection details under Settings, Project, Backend secrets, never in chat. Use the API service and the dev environment. Add `COLDTRACE_API_URL` (the Cloud Run address) and `COLDTRACE_INGEST_KEY` (the ingest key Moorthy and Peter issue). Secrets take effect on the next deploy. The agent will not accept a secret pasted into chat.
5. Choose Plan mode for the first message. Ask the agent to read the brief and propose a plan, then switch to Build and say "Build phase 1 of the plan."

Non-secret public values, such as the backend's public address, go in `app.json` under `expo.extra`. Never put a secret inside the app: anyone can extract values from a downloaded app.

First message, in Plan mode:

```
Read the attached ColdTrace build brief and AGENTS.md. Do not change any
files. Propose a plan that follows the Build order section, phase by phase.
List any part of the brief this platform cannot do, and what you would do
instead. Ask me before assuming anything about web support or the framework.
```

3. The prompt formula and when to use each mode

Every prompt asks for one change, names the screen, and says how you will know it is done.

Formula: **Context** (the screen title as it appears in the app) + **One change** + **Data and fields** + **States** (loading, empty, error) + **Done when** (what you will check).

Choose the mode before you send each message. Credits are charged by work done in every mode, so Plan and Ask are not discounted; they are cheaper only because they stop you building the wrong thing.

Mode	Use it for	Example opener
Plan	A big change, or the start of a phase. Writes a plan, changes no files	"Plan phase 3. Do not change any files."
Ask	Questions and diagnosis. Changes no files	"Don't change anything yet. Why does Case Review show an empty batch list?"
Build	Writing the code, fixing, styling	"Build the Alerts Queue screen from the plan."

When a bug survives two attempts, switch to Ask, request the cause, and only then switch to Build for the fix.

Do	Don't
Ask for one screen or one change per message	List ten features in one message
Say what you see and what you expected: "Tapping Acknowledge does nothing. I expected the alert to move to Acknowledged"	Say "it's broken"
Use the app's own screen titles and button labels	Describe a screen by position only
Attach a screenshot when something looks wrong	Describe visual problems in words alone
State the sign-in and data rules up front: "Each user signs in, and sees only what their role allows"	Leave sign-in open and let the agent guess
Use Restore to here, then send a clearer prompt	Argue with the agent across several messages

Do	Don't
Answer the agent's questions with an option, your own text, or Skip	Ignore a question and send a new request

4. Prompt library: ColdTrace in build order

Send these in order, one per message, and check the "Done when" line before sending the next. They refer to sections of the build brief: Design system, Roles and navigation, Screens, Data contract, Security and quality, Build order. Replace anything in angle brackets.

Phase 0: standing rules and plan (Plan mode)

```
Create AGENTS.md at the project root containing the block from Brief section
"How Newly should use this brief". Then read the whole brief and write a plan
for phases 1 to 7 from the Build order section. Do not build any screens yet.
Done when: AGENTS.md exists, and the plan lists each phase with the screens
it covers and any part of the brief you cannot do.
```

Phase 1: foundations (Build mode)

```
Build phase 1. Create the theme from Brief section "Design system" as named
tokens (colours, type scale, spacing, status colours, light and dark). Build
the navigation shell for the roles in "Roles and navigation": Operations
Lead, Reviewer, Auditor, Admin. Build the Login screen on Newly Backend email
sign-in, with a demo user for each role. All screens are empty placeholders
for now.
Done when: I can sign in as each role and see only that role's tabs, in light
and dark mode.
```

Phase 2: Command Center and Location Detail on mock data (Build mode)

```
Build phase 2. Create a data adapter with a mock implementation that returns
the sample data in Brief section "Data contract", behind the same function
names as the real API. Build Command Center and Location Detail exactly as
described in "Screens": three location tiles, alert feed, KPI strip, and the
temperature chart with the 2 to 8 degree band. Include loading, empty and
error states.
Done when: tapping a location tile opens its detail, and every state can be
shown from the mock.
```

Phase 3: Alerts Queue and Case Review, the full journey (Build mode)

```
Build phase 3 on the mock adapter. Build Alerts Queue with filters and sort, then Case Review with the evidence table, the evidence hash and a verified badge, rules and model verdicts, the incident note labelled as AI-written, and the actions Acknowledge, Confirm real, Mark false alarm, Add note, Escalate and Close with outcome and root cause. Every action writes an audit event.
Done when: I can take one alert from Open to Closed, and the audit list shows each step with who and when.
```

Phase 4: Batches at Risk, Detection Quality, Audit and Evidence (Build mode)

```
Build phase 4 on the mock adapter. Add Batches at Risk (product sensitivity, estimated freshness, spoilage risk, remaining window), Detection Quality (event recall, time to detect, false alerts per location-day for threshold, rules and random forest, plus the NIM comparison table), and Audit and Evidence (filterable list with hash verification). Show the synthetic-data banner on Detection Quality.
Done when: each screen matches "Screens" and is visible only to the roles listed there.
```

Phase 5: Settings and Access, Demo Simulator (Build mode)

```
Build phase 5. Add Settings and Access: read-only detection thresholds, notification channels, user and role list, and a connection status panel. Add Demo Simulator for the Admin role only: replay the sample readings, and inject one exception of each type at a chosen location. The simulator must be clearly labelled as demo mode.
Done when: the Admin can inject a cooling failure and watch it appear in the Command Center and Alerts Queue.
```

Phase 6: connect the real backend (Build mode)

```
Build phase 6. Replace the mock adapter with real calls that go through Newly Backend server code, which calls COLDTRACE_API_URL with the secret COLDTRACE_INGEST_KEY. The app must never hold or send that key. Keep the mock adapter available behind a demo flag that is off in production. Show how to test one call, and tell me which endpoints in Data contract are not built yet.
Done when: Command Center shows live detections from the ColdTrace backend, and a search of the app code finds no key.
```

Phase 7: hardening (Build mode)

```
Build phase 7. Apply Security and quality from the brief: accessibility
labels and contrast, text scaling, keyboard or focus order where supported,
offline and error states on every screen, session timeout, and role checks
enforced on the server as well as in the app. Add tests for the alert
lifecycle and the role rules. Write a README with the sections the brief
lists. Report anything not done.
Done when: the tests pass, and I can follow the README from a clean checkout.
```

5. Fixing problems: repair prompts

Fix a problem with one targeted prompt, and go back to an earlier version when two attempts have not worked.

Symptom	Prompt to send	Mode
The screen differs from the brief	`Compare the <screen title> screen with Brief section "Screens". List every difference. Do not change anything yet.` Then: `Fix the differences you listed.`	Ask, then Build
A bug survives two attempts	`Don't change anything yet. Explain the cause of: <what I see> when I expected <what I expected>.`	Ask
The agent invented data fields	`Use only the fields in Brief section "Data contract". Remove <field> and show me where it was used.`	Build
The agent added a new library	`Remove <library>. AGENTS.md says no new libraries without asking. Use what is already installed.`	Build
Wrong colours or type	`Use only the theme tokens from the Design system. Replace every hard-coded colour on <screen title> with a token.`	Build
A screen is visible to the wrong role	`<screen title> must be hidden from the <role> role. Enforce this in the server code, not only in the tab bar, and show me the check.`	Build

Symptom	Prompt to send	Mode
A secret was typed into chat	Do not send more prompts about it. Rotate the key at its source, set the new value under Backend secrets, and redeploy	None
The build keeps failing	Press Stop. Use Restore to here under the last message that worked, then send a smaller, clearer prompt	Build
Credits are draining in a repair loop	Press Stop. Automatic repair stays off until your next message. Ask: `Don't change anything. Why does the build fail?`	Ask

Three facts help here. When a build or crash happens, the agent repairs it automatically, and those turns use credits. In a Remote chat, the agent stops and explains after the same build failure three times in a row. Problems with sign-in or connecting an account need you; the agent will not repair them on its own.

6. Credits, versions, GitHub and the review checklist

Spend credits on small, checked steps, save every good state, and run the checklist before anyone else reviews the code.

Credits. They follow the work the agent does, not your message count. Quick questions cost a fraction of a credit; large builds and repair loops cost more. Previews, cloud simulators, native builds and Undo do not use credits. There is no spending cap, so watch the balance during phases 2 to 4. A single turn is limited to three hours, and a turn that hits the limit has no Retry button: send a new message.

Versions. Every turn that changes files is saved as a version, even if the build failed. Use Restore to here under a message to go back. Restoring changes `dev` for every chat in the project, so tell the team first.

Branches. Chats save to `dev`. Production runs `main`. To release: Settings, Project, Update main from dev, then Deploy backend to prod. Avoid `newly deploy --env prod`: it ships your local backend folder, not `main`.

GitHub, unresolved. The Newly pages disagree. One says the code can stay in step both ways with a repository you own. The FAQ says projects do not sync to GitHub. Check the Connect GitHub page, or ask Newly support, before depending on it. The fallback works either way: run `newly pull <project-id>` to get a copy of the code, push it to the team branch, and open a pull request to `main`. Leave the pull request open; the programme says do not merge it.

Review checklist before handing over

The app's reviewers are Peter (security), Moorthy (integration) and Chantell (data). Tick each item with evidence, such as a screenshot or a search result.

- [ ] No keys or secrets anywhere in the app code, `app.json` or the repository (Peter)
- [ ] Roles are enforced in server code, not only in the tab bar (Peter)
- [ ] Every action writes an audit event with who, what and when (Peter)
- [ ] The mock adapter is off in production (Moorthy)
- [ ] Each call matches the Data contract, and unbuilt endpoints are listed (Moorthy)
- [ ] The evidence hash is shown on each case, with a verified or mismatch badge (Peter, Chantell)
- [ ] The synthetic-data banner shows on Detection Quality (Chantell)
- [ ] Field names and units match the backend, such as degrees Celsius and 30-minute readings (Chantell)
- [ ] The README covers audience, problem, SDG, solution, setup, Google Cloud and NVIDIA integration, screenshots, team credits and individual contributions
- [ ] The code is on the team branch, with a pull request to `main` left open

As of 11 November 2026. Where the Newly documents were silent, this manual says so rather than guessing.

Sources

- Newly docs home
- Chatting with the agent
- Prompting tips
- Modes
- Where your code lives
- FAQ
- Backend overview
- Production and secrets
- AI app workflow guide